p0572R2
Static reflection of bit fields

Published Proposal, 5 May 2018

This version:
https://wg21.link/P0572r2

Author:
Alex Christensen (Apple)

Audience:
SG7

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
https://github.com/achristensen07/papers/blob/master/source/P0572r2.bs

---

Abstract

A proposal for the ability to determine where a bit field is stored within a byte.

Table of Contents

§ 1. Introduction

P0572r1 established the need to inspect size and location of bit fields, but its syntax was rejected by EWG in favor of putting the new functionality into reflection. This paper is the initial proposal exploring what this would look like.

§ 2. Additions to [d0194r6]

§ 2.1. Additions to section 21.11.2:

```
// 21.11.4.7 Member operations
… existing things …
template <RecordMember T> struct get_bit_size;
template <RecordMember T> struct get_bit_offset;
template <RecordMember T>
 constexpr auto get_bit_size_v = get_bit_size<T>::value;
template <RecordMember T>
 constexpr auto get_bit_offset_v = get_bit_offset<T>::value;
```

2.2. Additions to section 21.11.4.7:

All specializations of above templates shall meet the UnaryTypeTrait requirements ([meta.rqmts]) with a base characteristic of integral_constant<size_t>. The value of get_bit_size should be the number of bits in the representation of the RecordMember. The value of get_bit_offset should be the offset in bits of the RecordMember from the beginning of its Record.

3. Possible additional future work

It is unclear if a similar templates should be introduced into reflection for the size in bytes of RecordMembers and their offset in bytes. These values can already be determined by using the sizeof and offsetof keywords, so adding them to reflection could be elegant but redundant.

If we do add get_size and get_offset, they should probably not be limited to RecordMember because other things have size. Restricting get_bit_size and get_bit_offset to RecordMembers is ok because the intended use of them is to get the size of bit fields which cannot exist as anything but RecordMembers, but they can work with addressable RecordMembers, too.

A new concept called Addressable should probably also be added to be able to distinguish bit fields from RecordMembers that can be used with std::addressof.

A program should be ill-formed if get_pointer is used with a bit field.

4. Revision History

- r0 This was presented at the meeting in Kona in 2017 to LEWG. LEWG sees this as static reflection, the Reflection SG is therefore a better venue.
- r1 Updated audience, fixed minor typos.
- r2 Changed title and syntax to fit in reflection.

References

Informative References

[D0194R6]
Static Reflection. URL: https://wg21.link/d0194r6